🎯 Arbitrary Pose Roadnet Navigation

1. Course Content

In industrial logistics and service robot scenarios, the robot's destination (such as charging pile socket, transfer belt for handover, or shelf material picking port with specific orientation) is usually **not exactly on the road centerline**, and often requires the vehicle to precisely dock at a specific angle (such as facing the charging base directly or perpendicular to the production line).

If only "waypoint ID navigation" is used, the vehicle can only stop at the pre-drawn waypoints on the road network, which cannot meet the fine parking requirements for entering workstations or warehouse slots.

The function implemented in this lesson can be understood as: **"First take the highway main route, then precisely park into the warehouse."** The user only needs to provide an arbitrary target coordinate and vehicle heading direction. The vehicle first completes long-distance high-speed main route cruising through the topological road network. After reaching the nearest road network node near the target point, it smoothly exits the main road network and automatically adjusts the vehicle heading to precisely park at the target pose.

---

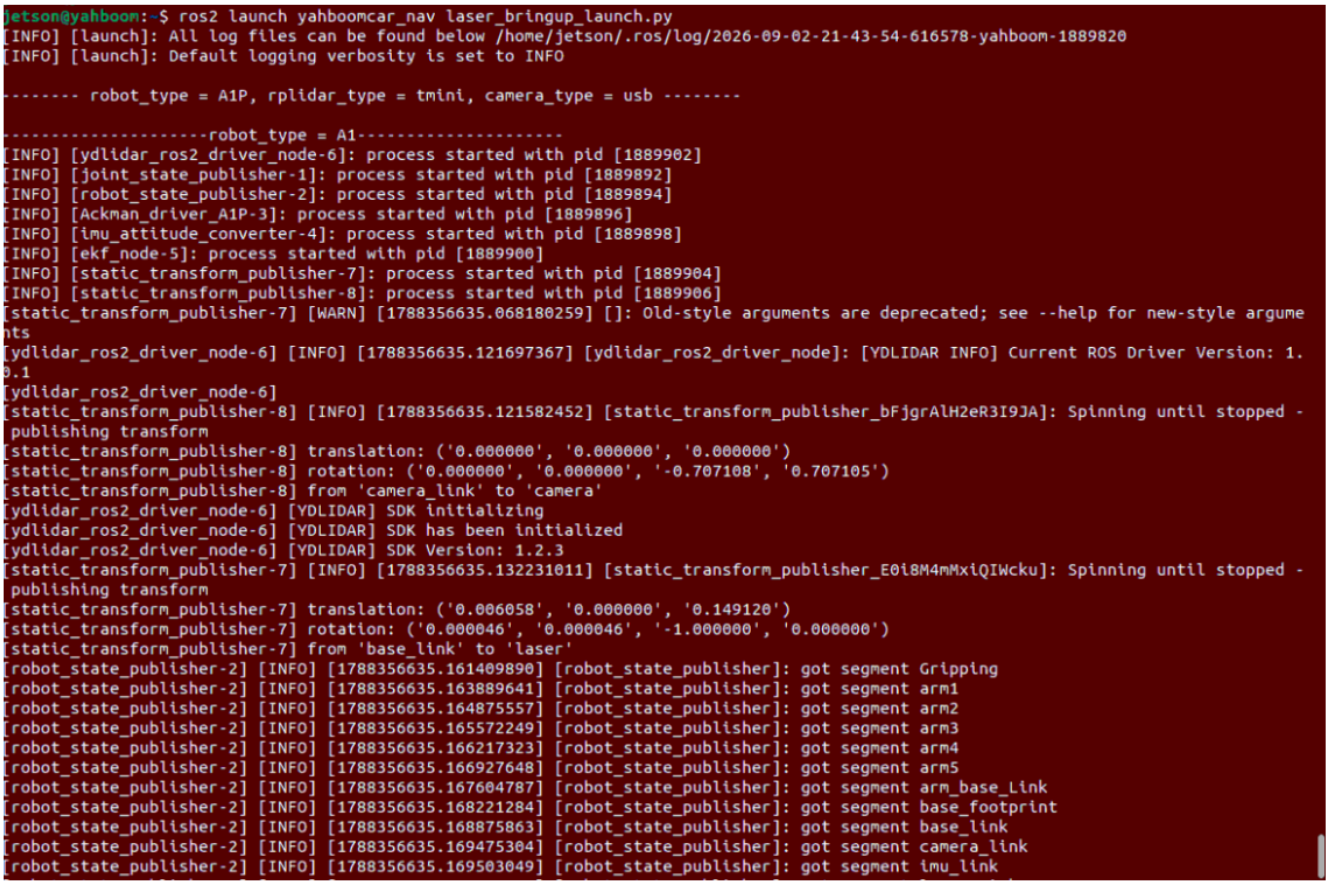
2. Preparation and Running

This case requires opening 4 terminals to start the system in sequence:

2.1 Start Chassis and LiDAR Driver (Terminal 1)

Run on the vehicle host:

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



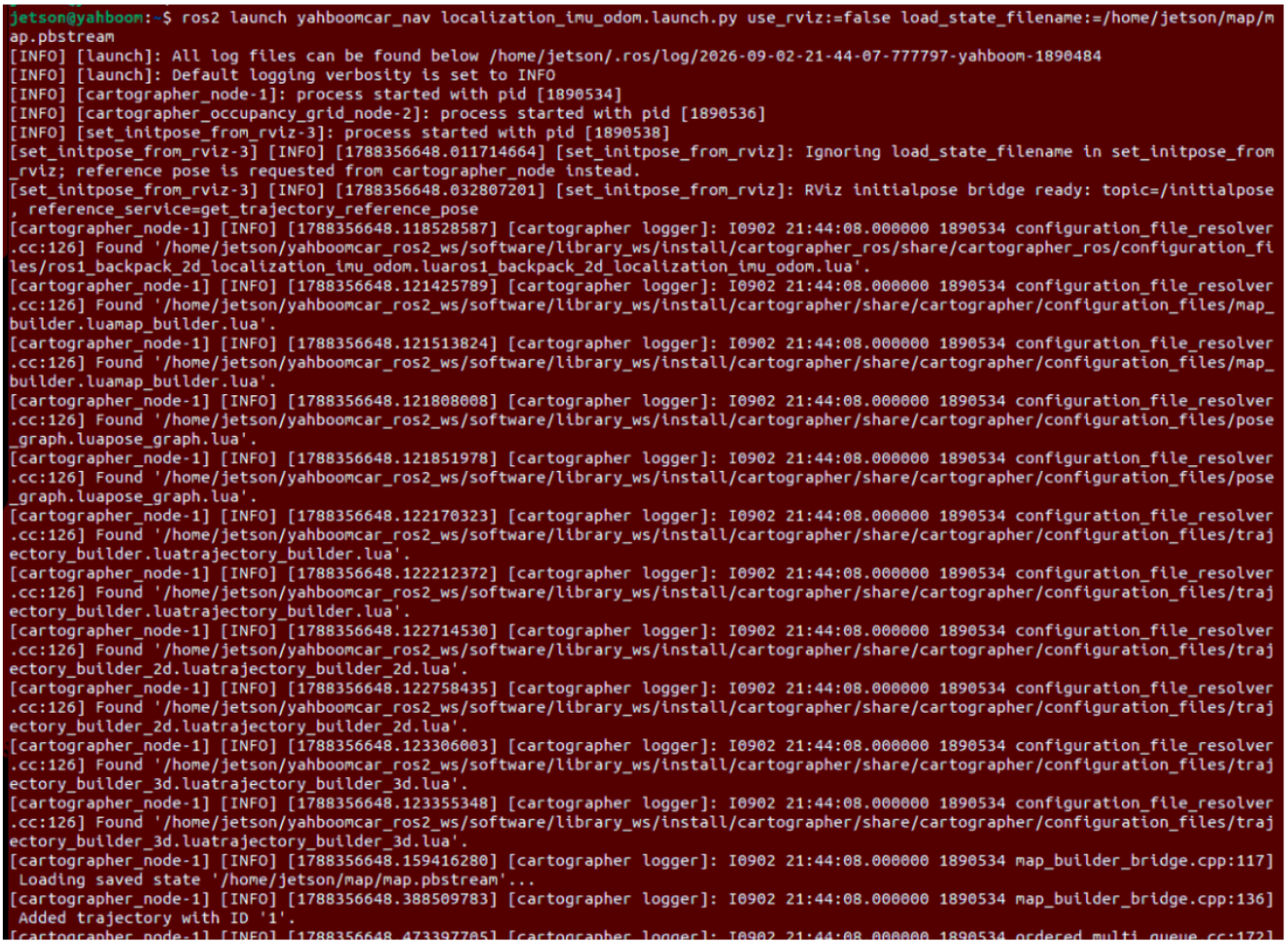
2.2 Start Cartographer Pure Localization Node (Terminal 2)

Load the built `.pbstream` pose map to provide high-precision global localization:

```
# RDK X5 system (sunrise user)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/sunrise/map/map.pbstream

# Or industrial PC jetson user
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/map/map.pbstream

# Raspberry Pi 5 user
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/root/map/map.pbstream
```



2.3 Start Roadnet Planning Navigation Core Control Node (Terminal 3)

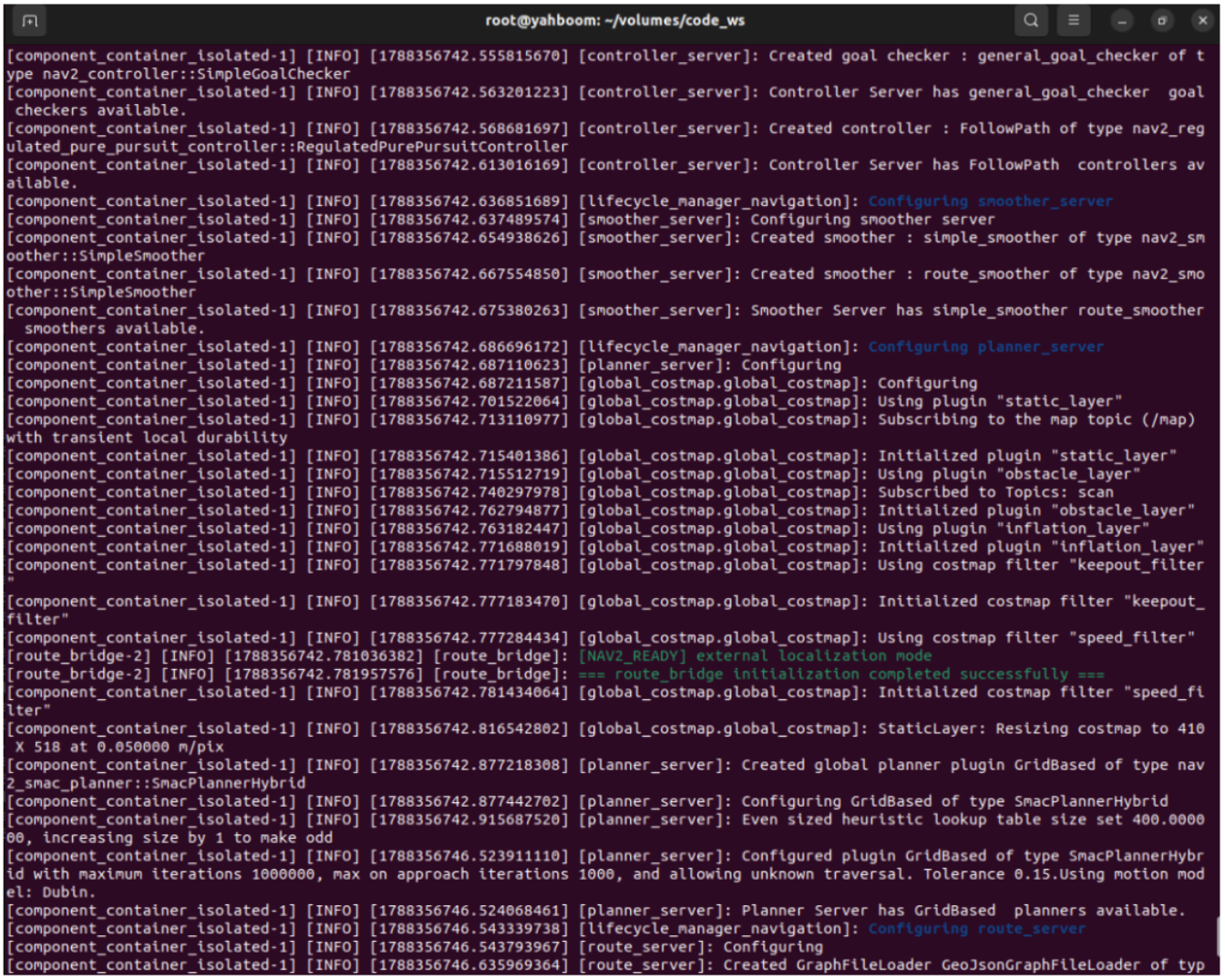
Enter the roadnet docker,

```
# Subsequent quick start command
bringup_ros2kilted
# Enter container terminal
exec_ros2kilted
```

This node automatically loads the `map.yaml` grid map, `map.geojson` topological road network, and starts the Nav2 path following controller and `route_bridge` orchestration engine:

```
# RDK X5 system
ros2 launch road_net_route roadnet_nav2.launch.py map:=map.yaml pose_map:=map
roadnet_file:=/home/sunrise/map/map.geojson

# If running in docker container
ros2 launch road_net_route roadnet_nav2.launch.py use_pose_precise_adjust:=true
map:=map.yaml pose_map:=map roadnet_file:=/root/map/map.geojson
```

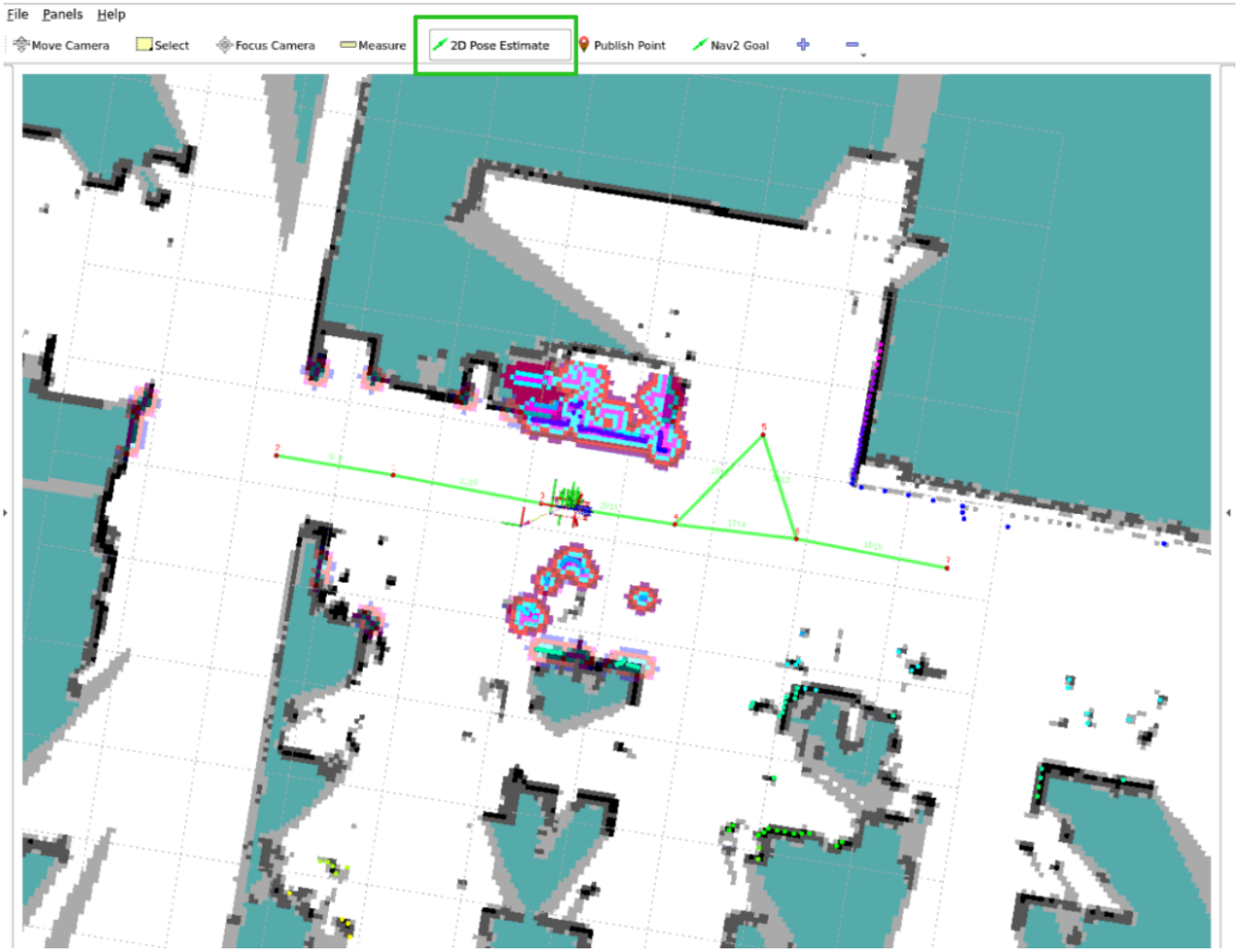


2.4 Start RViz Visualization (Terminal 4)

Run RViz monitoring window on the mainboard roadnet docker side: **(RDK and Raspberry Pi need to run on virtual machine)**

```
ros2 launch road_net_route rviz_only.launch.py
```

If localization has not converged, use the `2D Pose Estimate` tool in RViz to click and drag to align the initial pose:



3. Hands-on Demo and Pose Acquisition Methods

3.1 Getting Target Pose Parameters (Coordinates and Quaternion)

To make the vehicle navigate to a specified position and adjust to a specific heading, you first need to know the spatial coordinates and quaternion orientation of the target point.

- **You can manually place the vehicle at any desired position. If the pose has not converged, use the 2D Pose Estimate tool in RViz to click and drag, aligning the initial pose**

Run the following command in the terminal to print the vehicle's current pose in the `map` global coordinate frame in real time:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

The terminal output is as follows, where `Translation` corresponds to `position: {x, y}`, and `Rotation (Quaternion)` corresponds to `orientation: {z, w}`:

```
jetson@yahboom:~/map$ ros2 run tf2_ros tf2_echo map base_footprint
[INFO] [1788357972.864715949] [tf2_echo]: Waiting for transform map ->  base_footprint: Invalid frame ID "map" passed to canTransform arg
ument target_frame - frame does not exist
At time 1788357973.777632358
- Translation: [0.902, 2.893, -0.005]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 0.039, 0.999]
- Rotation: in RPY (radian) [0.000, -0.000, 0.078]
- Rotation: in RPY (degree) [0.000, -0.000, 4.461]
- Matrix:
  0.997 -0.078  0.000  0.902
  0.078  0.997  0.000  2.893
  0.000  0.000  1.000 -0.005
  0.000  0.000  0.000  1.000
At time 1788357974.771683782
- Translation: [0.900, 2.894, -0.005]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 0.039, 0.999]
- Rotation: in RPY (radian) [0.000, -0.000, 0.078]
- Rotation: in RPY (degree) [0.000, -0.000, 4.472]
- Matrix:
  0.997 -0.078  0.000  0.900
  0.078  0.997  0.000  2.894
  0.000  0.000  1.000 -0.005
  0.000  0.000  0.000  1.000
^C[INFO] [1788357975.610906592] [rclcpp]: signal_handler(signum=2)
```

3.2 Trigger Arbitrary Pose Roadnet Navigation

- **Publish to `/road_net_nav` topic directly via terminal command line**

Fill in the actual coordinates and heading obtained in the previous step into the topic parameters:

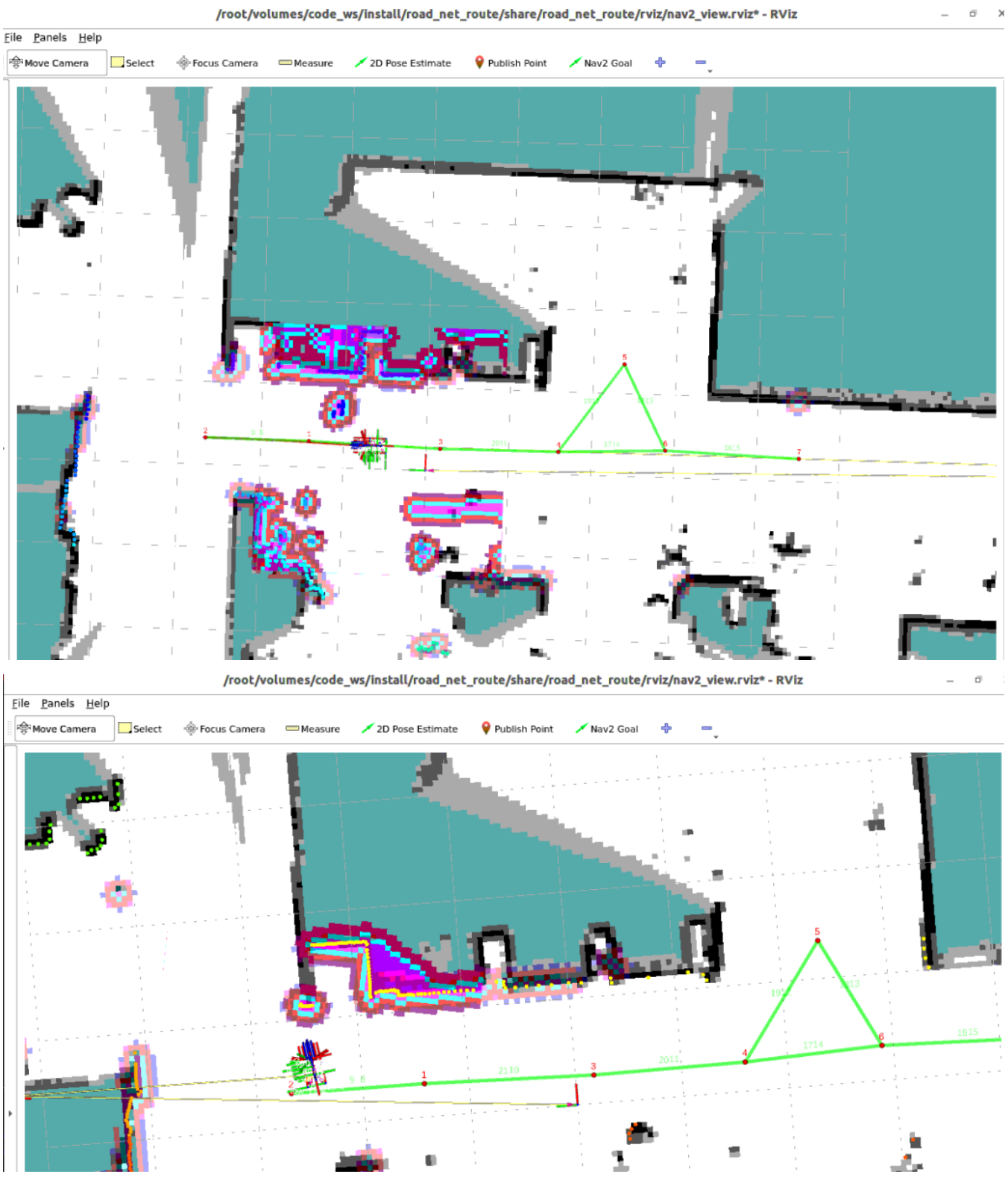
```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  header: {frame_id: 'map'},
  pose: {
    position: {x: 0.902, y: 2.893, z: -0.005},
    orientation: {x: 0.0, y: 0.0, z: 0.039, w: 0.999}
  }
}"
```

```
jetson@yahboom:~/map$ ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  header: {frame_id: 'map'},
  pose: {
    position: {x: 0.902, y: 2.893, z: -0.005},
    orientation: {x: 0.0, y: 0.0, z: 0.039, w: 0.999}
  }
}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.PoseStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=0, nanosec=0), frame_id='ma
p'), pose=geometry_msgs.msg.Pose(position=geometry_msgs.msg.Point(x=0.902, y=2.893, z=-0.005), orientation=geometry_msgs.msg.Quaternion(x
=0.0, y=0.0, z=0.039, w=0.999)))

jetson@yahboom:~/map$
```

After the vehicle starts, observe the RViz display:

1. The vehicle first drives smoothly within the roadnet topological lane;
2. After approaching the target, it exits the main road and drives toward the offline target point;
3. Upon arriving at the position, the vehicle fine-tunes the body heading in place until it completely matches the target quaternion:



After successful navigation, the system publishes `"road_net_nav_succeeded"` on the `/action_feedback` topic.

---

4. Core Source Code Analysis: Pose Navigation and End Maneuver Alignment

Source code reference:

- `code_ws/src/road_net_route/road_net_route/route_bridge.py`

- `code_ws/src/road_net_route/road_net_route/geom.py`
- `code_ws/src/road_net_route/road_net_route/maneuver.py`

In `route_bridge.py`, the arbitrary pose navigation fully implements the entire process of "**direction-aware topological matching + roadnet main route cruising + end pose precise alignment**" from any starting point to any ending point.

4.1 Target Pose Reception and Direction-Aware Node Matching

When receiving the `/road_net_nav` topic, the system first obtains the vehicle's current real-time TF pose, and matches the roadnet start and end nodes respectively:

```
# route_bridge.py —— nav_callback and direction-aware node matching
def nav_callback(self, msg: PoseStamped):
    """Pose navigation callback: receives target pose PoseStamped and pushes into event
queue for async processing"""
    self._log.log('info', '\033[95m', 'ROUTE_RX_POSE',
                  f"/road_net_nav frame={msg.header.frame_id or 'EMPTY'}, "
                  f"goal=({msg.pose.position.x:.3f},{msg.pose.position.y:.3f})")
    self._enqueue_navigation_event(msg)

def _find_nearest_current_node(self, fallback=None, enforce_threshold: bool = True):
    """Direction-aware nearest node selection:
    Not only calculates Euclidean geometric distance between vehicle and various waypoints,
but also performs heading projection evaluation based on current vehicle heading.
    Prioritizes waypoints directly in front of the vehicle body as entry points, applies
distance penalty to waypoints behind the vehicle body,
    completely avoiding sharp steering at startup or unnecessary reverse entry into the
network.
    """
    pose = self._get_current_pose()
    # Combine geometric distance and angular error to select the smoothest front entry node
as start_id
    ...
```

4.2 End Point Roadnet Exit Matching and Continuous Path Generation

The system calculates which roadnet node is nearest to the goal pose `goal_pose`, and uses it as the exit node for roadnet main route cruising:

1. First uses roadnet Dijkstra algorithm to generate topological main route from current nearest front node to target exit node;
2. Dense interpolation along lane polylines generates smooth continuous trajectory (`nav_msgs/msg/Path`), driving the chassis to advance at full speed along the roadnet;
3. During movement, continuously monitors Euclidean distance to target exit node (`id_goal_reached_tolerance = 0.2m`).

4.3 End Pose Precise Alignment

When the vehicle arrives at the target exit node, the roadnet main route cruising ends. The system automatically connects to end pose maneuver handling:

```
# route_bridge.py —— Arriving at target exit and end adjustment
if self._is_node_reached(goal_node_id, self.id_goal_reached_tolerance, 'GOAL_NODE'):
    self._log.log('info', '\033[32m', 'ROADNET_GOAL', 'Main route completed. Entering fine
adjustment...')

    # For workstation points exiting the roadnet centerline, execute end translation and in-
place rotation alignment
    if self.pose_precise_adjust_enabled:
        self.pose_precise_goal_pub.publish(goal_pose)
    else:
        # Or have maneuver controller drive vehicle rotation to align target yaw angle
        self.maneuvers.rotate_to_orientation(goal_pose.pose.orientation)

    # Finally publish task completion signal
    self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
```

This "**macro topology strict constraint + micro free parking adjustment**" architecture not only prevents vehicles from wandering illegally in factory main passages, but also meets the millimeter-level precise vehicle heading alignment requirements for various material picking stations and charging piles.